```
    .Include(c => c.Country)
    .Include(c => c.Properties);
```

You want to return all cities, including their related country and properties, so you can display details on the home page of the application. However, you only want to include currently available properties, so you apply the filter to the `Include` method:

```
var cities = _context.Cities
    .Include(c => c.Country)
    .Include(c => c.Properties.Where(p => p.AvailableFrom < DateTime.Now));
```

The next step is updating the Index.cshtml.cs file content and replacing the existing content with the following code, which injects the `ICityService` into the constructor and uses it to obtain city data.

Listing 8.15 Revised `IndexModel` code for the home page

```
using CityBreaks.Models;
using CityBreaks.Services;
using Microsoft.AspNetCore.Mvc.RazorPages;

namespace CityBreaks.Pages
{
    public class IndexModel : PageModel
    {
        private readonly ICityService _cityService;

        public IndexModel(ICityService cityService)
        {
           _cityService = cityService;
        }

        public List<City> Cities { get; set; }
        public async Task OnGetAsync() => Cities =
        ➥ await _cityService.GetAllAsync();
    }
}
```

Then you remove the list box from the home page (if it is still there from chapter 5) and replace it with the following.